Public - عام

Lab 5 Intents and Navigation Report

Guided Tasks 1 to 4 and StudyPlanner Semester App Part 2

Elaf Hassan Al-Nutayf

Student ID 2240005931 | Section 01

CSC 402 Mobile Application Programming
Computer Science Department
College of Science and Humanities in Jubail
Imam Abdulrahman Bin Faisal University

First Semester 2026–2027 | Report date 18 October 2026

Overview

In this lab, I connected the Lab05Courses screens using typed routes, opened the browser and phone dialer through implicit intents, and shared a selected course through the Android share sheet. I also tested two intentional failures and used Logcat to identify their causes. I then extended my StudyPlanner app with a task detail screen, navigation, Back, and task sharing.

The final navigation passes the selected course code or task title to the detail screen. Screen composables display state and report events through callbacks, while the app and Route composables handle navigation and intents. This report contains the completed task answers, key implementation code, test results, planning diagrams, and screenshot evidence.

Development environment

Item	Configuration
Projects	Lab05Courses and StudyPlanner
Language and UI	Kotlin with Jetpack Compose and Material 3
Build configuration	Kotlin DSL; minimum SDK 26; target SDK 37
Navigation setup	Navigation Compose 2.10.2; serialization plugin 2.2.10
Test device	Pixel 10 emulator shown in Android Studio; API 37.1

Report guide

Pages 2–5: guided task answers and implementation.
Pages 6–9: StudyPlanner Part 2 design, implementation and testing.
Page 10: AI use disclosure, reflection and reference.
Pages 11–28: full screenshot evidence E01–E18.

Public - عام

Task 1 Ask another app

Steps 1 1 to 1 3 Events and safe intents

The HomeScreen event parameters are onWebsiteClick, onCallClick and onCoursesClick. I used LocalContext.current in HomeRoute and kept the external app actions inside their click callbacks. The missing dialer callback was implemented as follows:

```
onCallClick = {
    val intent = Intent(
        Intent.ACTION_DIAL, Uri.parse("tel:0138000000")
    )
    try {
        context.startActivity(intent)
    } catch (e: ActivityNotFoundException) {
        Toast.makeText(
            context, "No app can open this", Toast.LENGTH_SHORT
        ).show()
    }
}
```

For the initial test, MainActivity called HomeRoute(onCoursesClick = {}) inside Lab05CoursesTheme. The Browse courses callback was empty at this stage. The home screen, browser and dialer are shown in E01–E03.

Step 1 4 Button results

Button	Action	Data URI	Scheme	App opened
University website	ACTION_VIEW	https://www.iau.edu.sa	https	Chrome
Call the department	ACTION_DIAL	tel:0138000000	tel	Phone dialer

The dialer displayed the placeholder number 0138000000. Opening the dialer did not place a phone call.

Step 1 5 Unsupported address with error handling

After I changed the website URI to csc402://home, the app displayed the Toast “No app can open this” and stayed open. No installed activity could handle the custom scheme. The catch block handled ActivityNotFoundException, so the failure did not terminate the app. Evidence E04 shows the Toast and the guarded code.

Step 1 6 Unsupported address without error handling

After removing the website try/catch block, tapping the same button caused the app to stop. Logcat reported android.content.ActivityNotFoundException with the message “No Activity found to handle Intent”. The failing intent used ACTION_VIEW and csc402://home. E05 shows the crash dialog; E06 shows the exception and the call at CourseScreens.kt:113.

Step 1 7 Restore and retest

I restored https://www.iau.edu.sa and the try/catch block. The university website opened again in Chrome, as shown in E07. The final HomeRoute protects both the website and dialer launches.

Public - عام

Task 2 Plan screens and routes

Steps 2 1 to 2 3 Screen decisions

The app has three screens: Home, Course list and Course detail. Course detail needs the selected course code to open, so its route carries code: String. Home and Course list need no arguments and use object routes. Only the small identifying value is passed between destinations.

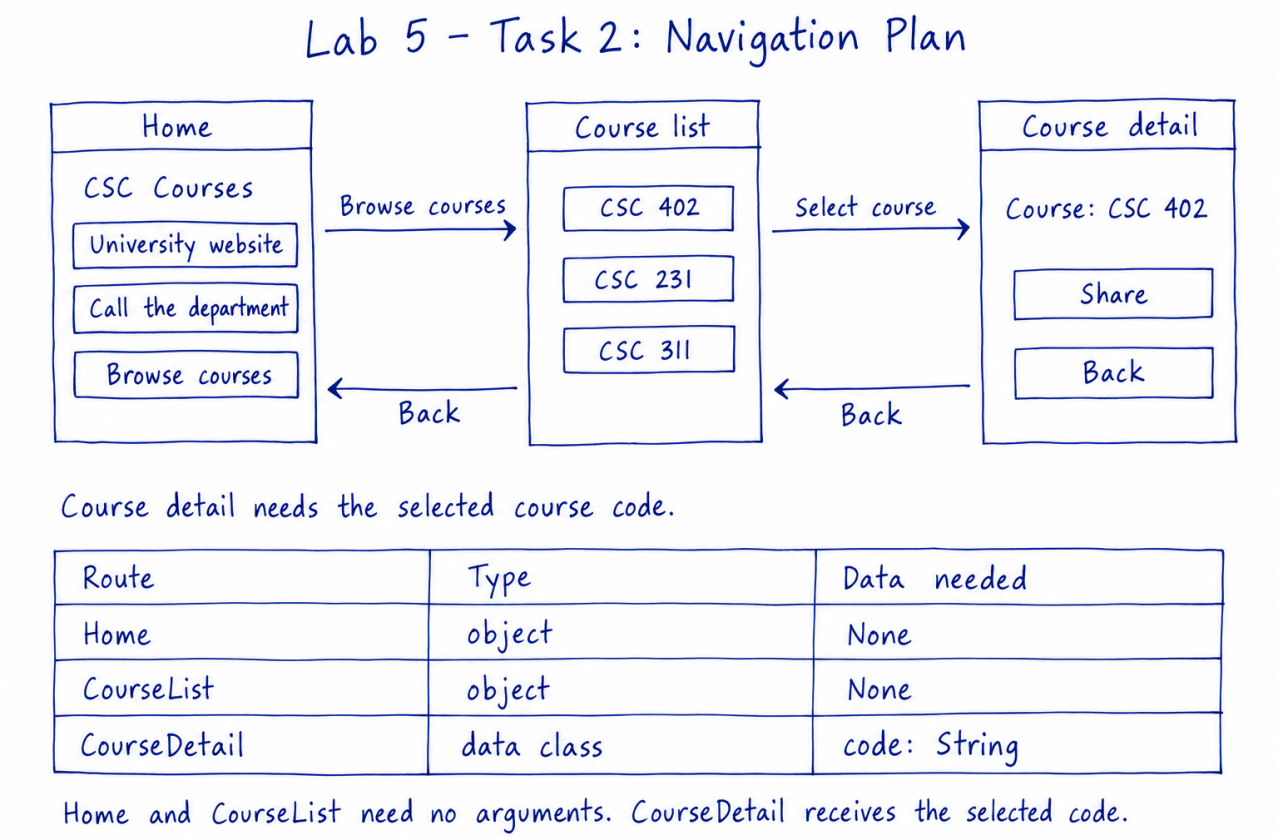


Figure 1. My course browser screen map and route plan.

Step 2 4 Navigation setup

I added the following entries to the app module Gradle file and synced the project. The serialization plugin version matches kotlin = "2.2.10" in the version catalog.

```
// In plugins
kotlin("plugin.serialization") version "2.2.10"
// In dependencies
implementation("androidx.navigation:navigation-compose:2.10.2")
```

Step 2 5 Route declarations

The final navigation/Routes.kt contains these declarations and imports kotlinx.serialization.Serializable:

```
@Serializable object Home
@Serializable object CourseList
@Serializable data class CourseDetail(val code: String)
```


Public - عام

Public - عام

Task 3 Connect the screens

Steps 3 1 and 3 2 Navigation graph

CourseApp owns the NavController. Its NavHost starts at Home, moves to the list, and reads the selected code inside the detail destination. The completed navigation body is:

```
val navController = rememberNavController()

NavHost(navController = navController, startDestination = Home) {
    composable<Home> {
        HomeRoute(
            onCoursesClick = { navController.navigate(CourseList) }
        )
    }
    composable<CourseList> {
        CourseListRoute(
            onCourseClick = { code ->
                navController.navigate(CourseDetail(code))
            }
        )
    }
    composable<CourseDetail> { backStackEntry ->
        val detail: CourseDetail = backStackEntry.toRoute()
        CourseDetailRoute(
            code = detail.code,
            onBackClick = { navController.popBackStack() }
        )
    }
}
```

MainActivity now displays the navigation graph inside the existing theme:

```
setContent {
    Lab05CoursesTheme {
        CourseApp()
    }
}
```

Step 3 3 Passing the selected code

After I tapped Browse courses and then CSC 231, the detail screen displayed CSC 231. The line navController.navigate(CourseDetail(code)) carried the selection. The destination recovered it with backStackEntry.toRoute() and supplied detail.code to CourseDetailRoute. E08 shows the three courses; E09 shows the selected course detail.

Step 3 4 Device Back

Using device Back twice from Course detail returned first to Course list and then to Home. Navigation Compose handled the system Back event through the navigation back stack. I did not add a separate device Back handler.

Step 3 5 Screen responsibilities

CourseListScreen does not receive or use a NavController. Its button calls onCourseClick(code). CourseListRoute forwards that event, and CourseApp decides the destination. This keeps the screen independent of navigation and allows it to be reused and previewed with simple callbacks.

Public - عام

Task 4 Sharing and the back stack

Steps 4 1 and 4 2 Share a selected course

I implemented sharing inside CourseDetailRoute. The selected code is included in EXTRA_TEXT:

```
val send = Intent(Intent.ACTION_SEND).apply {
    putExtra(Intent.EXTRA_TEXT, "I'm taking $code this semester!")
    type = "text/plain"
}
context.startActivity(Intent.createChooser(send, "Share via"))
```

The sheet showed Chrome and Google Drive, among other targets. E10 displays “I'm taking CSC 231 this semester!” and E11 displays “I'm taking CSC 402 this semester!”.

The lab uses a chooser instead of launching the ACTION_SEND intent directly. Android opens its system share interface and presents the compatible receivers. This avoids directly requesting a receiving app that may not exist; on the tested emulator, the system chooser was available and opened successfully.

Step 4 3 Back stack trace

The table lists screen destinations from bottom to top. The rightmost destination is the visible screen.

Step	Action	Back stack after the action
1	The app opens	Home
2	Tap Browse courses	Home → CourseList
3	Tap CSC 402	Home → CourseList → CourseDetail("CSC 402")
4	Tap the on-screen Back button	Home → CourseList
5	Tap CSC 231	Home → CourseList → CourseDetail("CSC 231")

Step 4 4 Remove the serialization annotation

I removed only @Serializable above CourseDetail and kept the import. Android Studio did not show a red editor error for that declaration in the captured state. The app built, but crashed when it opened and the navigation graph was created, before I could select a course. The exception line in Logcat was:

```
kotlinx.serialization.SerializationException: Serializer for class
'CourseDetail' is not found.
```

Logcat then asked for @Serializable and the serialization compiler plugin. The stack included NavDestinationBuilder initialization. Without the annotation, the serializer required for the typed CourseDetail route was missing. Evidence E12 records this intentional failure.

Step 4 5 Restore and confirm

I restored @Serializable above CourseDetail and reran the app. Navigation, sharing and Back worked again. The final Routes.kt contains @Serializable on all three routes.

Public - عام

StudyPlanner Part 2 Screen plan and routes

P2 1 Task detail design

The task detail screen shows the selected task title. It requires title: String to open. The user can share that title or return to Home. The map has two destinations: Home → Task detail, with Back returning to Home.

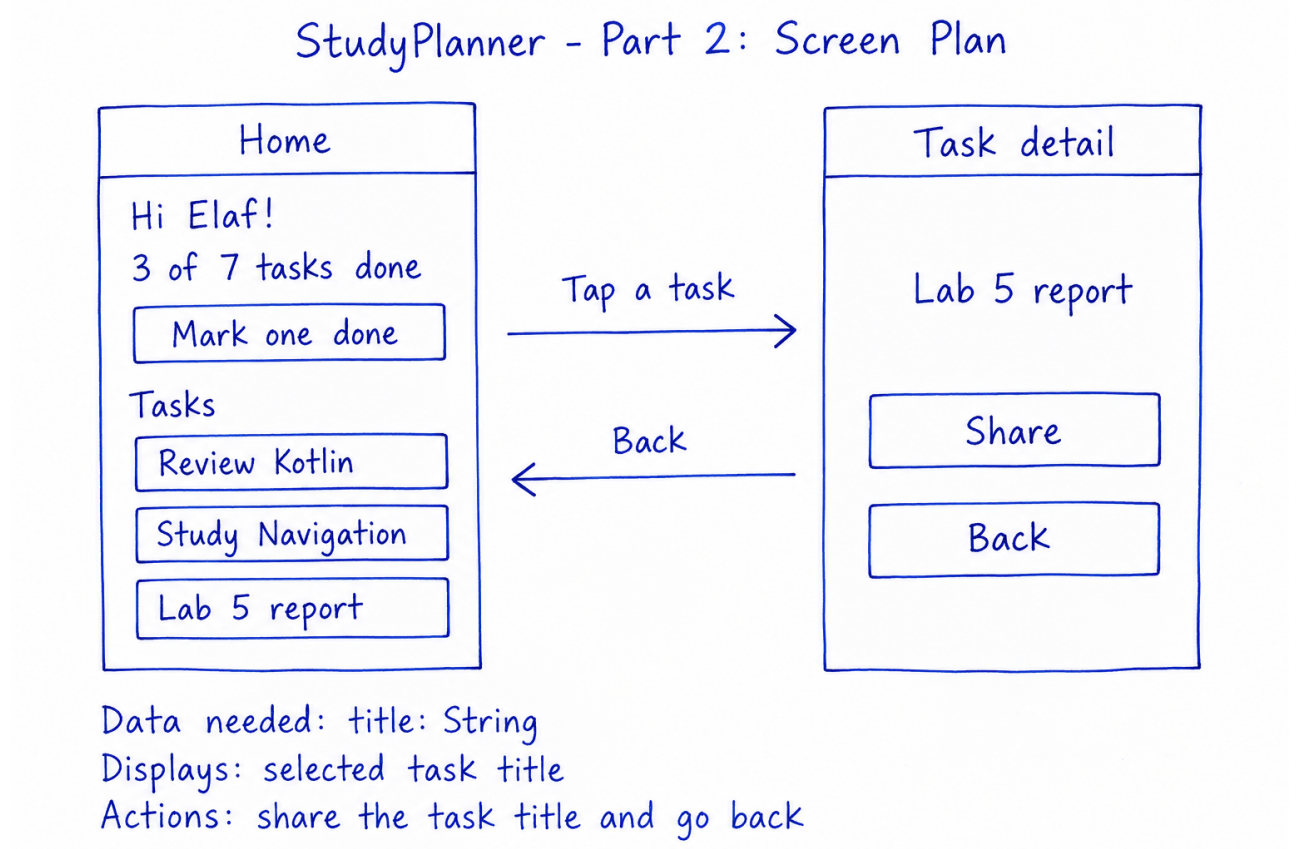


Figure 2. My StudyPlanner screen and navigation plan.

P2 2 Project setup

I continued in the existing StudyPlanner project. I added Navigation Compose 2.10.2 and kotlin("plugin.serialization") version "2.2.10" in the app module. The app still uses one Activity and the existing StudyPlannerTheme.

P2 3 Typed routes

The routes are in the sa.edu.iau.cshj.studyplanner.navigation package and use kotlinx.serialization.Serializable:

```
@Serializable
object PlannerHome

@Serializable
data class TaskDetail(val title: String)
```

PlannerHome needs no argument. TaskDetail carries the selected title, so different buttons can open the same detail layout with different data.

Public - عام

StudyPlanner Part 2 Home and detail screens

P2 4 Task buttons on Home

I added tasks to PlannerUiState and initialized the Route with Review Kotlin, Study Navigation and Lab 5 report. The existing greeting and progress values remain in the state:

```
data class PlannerUiState(
    val greeting: String = "Hi Elaf!",
    val totalTasks: Int = 7,
    val doneTasks: Int = 3,
    val tasks: List<String> = emptyList()
)
```

PlannerHomeScreen accepts onTaskClick: (String) -> Unit and reports the tapped title:

```
uiState.tasks.forEach { title ->
    Button(onClick = { onTaskClick(title) }) { Text(title) }
}
```

PlannerHomeRoute owns the mutable state. Mark one done updates a copy of the state only while doneTasks is below totalTasks. E13 shows the Home screen during P2.4; E18 shows it after the navigation graph was connected.

P2 5 Stateless detail screen and previews

TaskDetailUiState holds title with an empty-string default. The detail composable receives data and two event callbacks, with a default modifier last. It does not own mutable state, create an Intent, or receive a NavController.

```
data class TaskDetailUiState(val title: String = "")

@Composable
fun TaskDetailScreen(
    uiState: TaskDetailUiState,
    onShareClick: () -> Unit,
    onBackClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(uiState.title, style = MaterialTheme.typography.headlineMedium)
        Button(onClick = onShareClick) { Text("Share") }
        OutlinedButton(onClick = onBackClick) { Text("Back") }
    }
}
```

I added two previews wrapped in StudyPlannerTheme: TaskDetailReportPreview uses “Lab 5 report”, and TaskDetailKotlinPreview uses “Review Kotlin”. Each supplies empty event callbacks. Both rendered successfully in Android Studio, as shown together in E14.

Public - عام

StudyPlanner Part 2 Sharing and navigation

P2 6 Share from the Route

TaskDetailRoute receives title and onBackClick. It reads LocalContext.current and passes TaskDetailUiState(title = title) to the screen. The onShareClick callback contains:

```
val send = Intent(Intent.ACTION_SEND).apply {
    putExtra(Intent.EXTRA_TEXT, "Today I'm studying: $title.")
    type = "text/plain"
}
context.startActivity(Intent.createChooser(send, "Share via"))
```

The Route forwards onBackClick to the screen. E16 shows the share sheet containing “Today I'm studying: Lab 5 report.”

P2 7 Connect Home to Task detail

StudyPlannerApp owns the NavController. The graph passes the tapped title into TaskDetail, reads it back with toRoute(), and pops the current destination for the on-screen Back button:

```
@Composable
fun StudyPlannerApp() {
    val navController = rememberNavController()
    NavHost(
        navController = navController,
        startDestination = PlannerHome,
        modifier = Modifier.safeDrawingPadding()
    ) {
        composable<PlannerHome> {
            PlannerHomeRoute(
                onTaskClick = { title ->
                    navController.navigate(TaskDetail(title))
                }
            )
        }
        composable<TaskDetail> { backStackEntry ->
            val detail: TaskDetail = backStackEntry.toRoute()
            TaskDetailRoute(
                title = detail.title,
                onBackClick = { navController.popBackStack() }
            )
        }
    }
}
```

MainActivity calls the graph inside the theme. I also added its import from the navigation package.

```
setContent {
    StudyPlannerTheme {
        StudyPlannerApp()
    }
}
```

safeDrawingPadding keeps the content clear of the system bars. E15 shows Lab 5 report, E17 shows Review Kotlin, and E18 shows the connected Home screen.

Public - عام

Public - عام

StudyPlanner Part 2 Verification

Manual test results

I ran the app in the emulator and checked the flow from Home to details, sharing, and returning Home. The selected task title changed with the button I tapped.

Test	Observed result	Evidence
Open Home	Hi Elaf!, progress, and the three task buttons appear.	E13 and E18
Tap Lab 5 report	The detail title is Lab 5 report with Share and Back.	E15
Tap Share	The system sheet contains Today I'm studying: Lab 5 report.	E16
Close the share sheet	The task detail screen is available again.	Manual check
Tap on-screen Back	The app returns to PlannerHome.	Manual check
Tap Review Kotlin	The same detail screen now displays Review Kotlin.	E17
Use device Back	The app returns to PlannerHome.	Manual check
Render both detail previews	Both titles render with Share and Back.	E14

Implementation requirements

Requirement	Implementation
State is passed into screens	PlannerHomeScreen receives PlannerUiState; TaskDetailScreen receives TaskDetailUiState. Neither screen declares remember or mutableStateOf.
Events flow out through callbacks	Home reports onTaskClick(title). Detail reports onShareClick and onBackClick.
Default modifier is last	Both screen functions end with modifier: Modifier = Modifier.
Typed routes carry small arguments	PlannerHome is an object. TaskDetail is a data class containing title: String. Both use @Serializable.
Navigation is outside the screens	StudyPlannerApp owns the controller. navigate is called from onTaskClick, and on-screen Back calls popBackStack.
Intents are created in a Route	TaskDetailRoute constructs ACTION_SEND inside onShareClick, with EXTRA_TEXT, text/plain and a chooser.
One Activity and Material 3	The manifest declares MainActivity only. UI components use androidx.compose.material3.

The implementation keeps display logic separate from platform actions. The previews can therefore render the detail screen using sample state without starting navigation or opening another app.

Public - عام

Public - عام

AI use disclosure and reflection

AI use and disclosure log for Lab 5

Course week: Week 5. Reported on 18 October 2026. Tools used: ChatGPT and Codex, including AI image generation for the planning diagrams.

Implementation guidance. I used AI assistance for step-by-step explanations and Kotlin code suggestions for implicit intents, typed routes, NavHost, toRoute(), sharing, and the StudyPlanner detail screen. I applied the changes in Android Studio and ran the apps to check the resulting screens and actions.

Debugging and changes afterwards. I used the assistant to interpret screenshots and errors. I added the required navigation import in MainActivity, corrected an extra closing brace, restored @Serializable after the intentional failure test, and used safeDrawingPadding for the StudyPlanner navigation content. I then retested task selection, Share and Back and captured the results.

Planning diagrams. I prepared the course navigation plan and StudyPlanner screen plan using AI image generation. Figures 1 and 2 show the screens, navigation flow and route arguments for my applications.

Report assistance. AI helped draft and organize the explanations, answer tables, code excerpts and screenshot captions, and format this report. The emulator and Android Studio screenshots document the work I ran. The intentional failure screenshots are labeled separately from the final working behavior.

What I learned

I learned how an implicit intent lets Android choose an app based on an action and data. Testing csc402://home showed why an unavailable receiver must be handled. I also learned how typed routes carry a small value to a destination and why @Serializable is necessary even when a route declaration does not show an immediate editor error.

The back-stack trace helped me distinguish adding a destination from returning to a previous one. In StudyPlanner, I used the same detail layout for different tasks by passing the selected title. Keeping navigation and intents outside the screen composables also made the previews easier to write.

Reference and evidence

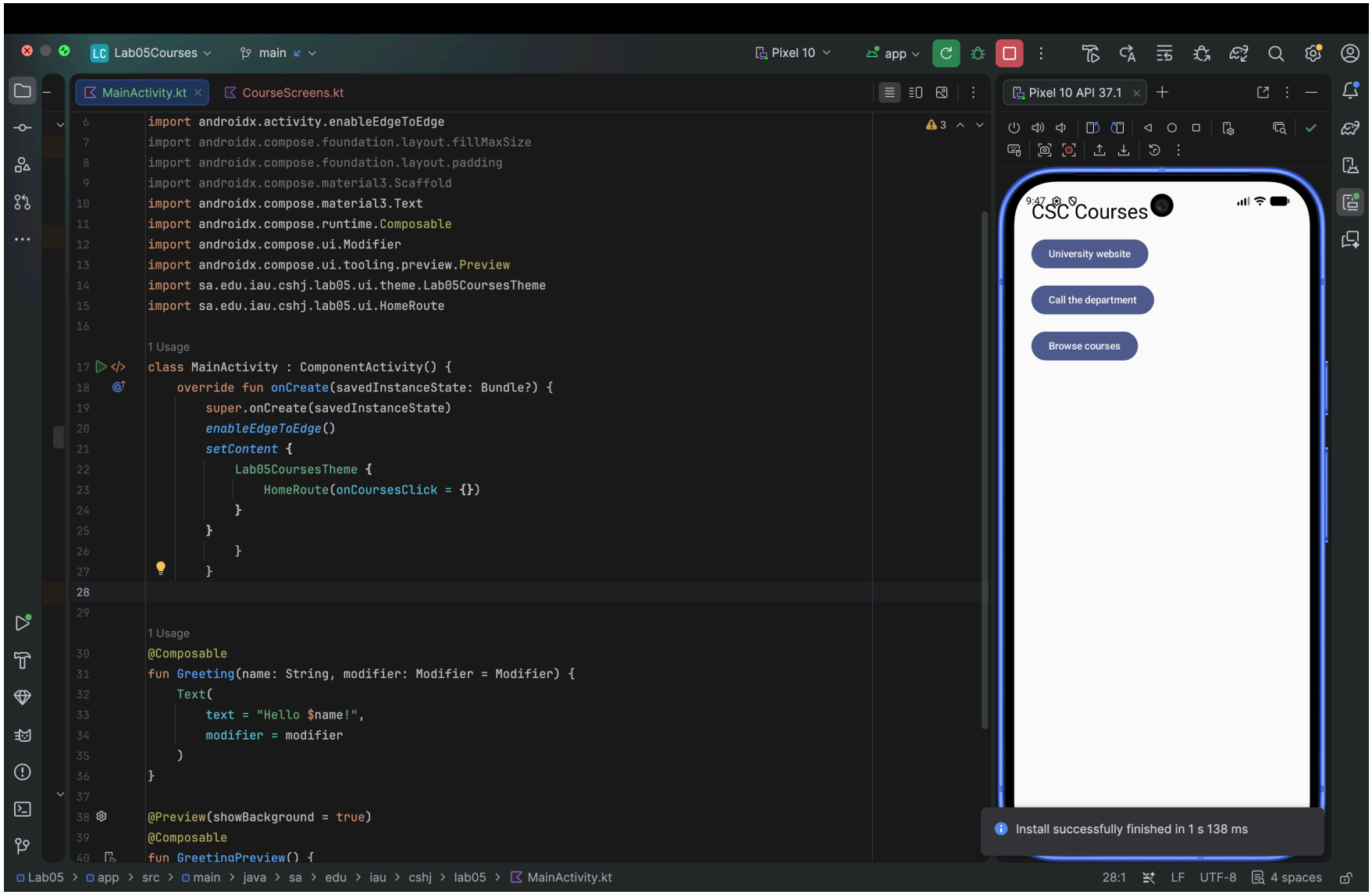
CSC 402. Lab 5 — Leaving the Screen: Intents and Navigation. Student Exercise File, First Semester 2026–2027. Computer Science Department, College of Science and Humanities in Jubail, Imam Abdulrahman Bin Faisal University. Guided Tasks 1–4 and Semester App Part 2, pages 1–11; starter code, pages 14–16.

Implementation excerpts correspond to the local Lab05 and StudyPlanner project files. Appendix evidence E01–E17 contains the seventeen task screenshots; E18 adds the final connected StudyPlanner Home screenshot. The two planning diagrams appear with the related design answers.

Public - عام

E01 Task 1 Initial Home screen

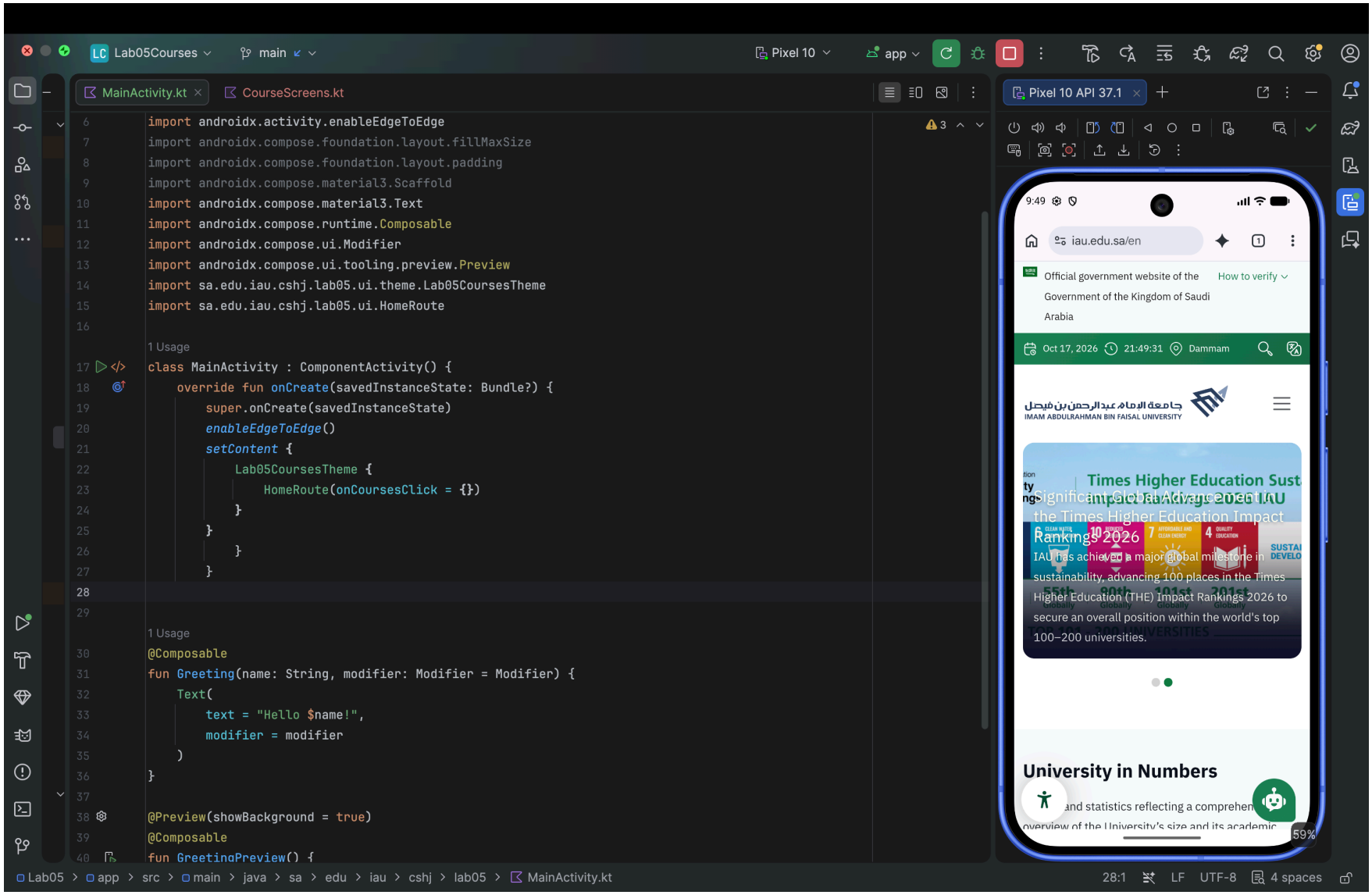
HomeRoute is displayed inside the theme. The three buttons are visible; Browse courses is still an empty callback at this stage.



E01. Task 1.3.png

E02 Task 1 Website intent result

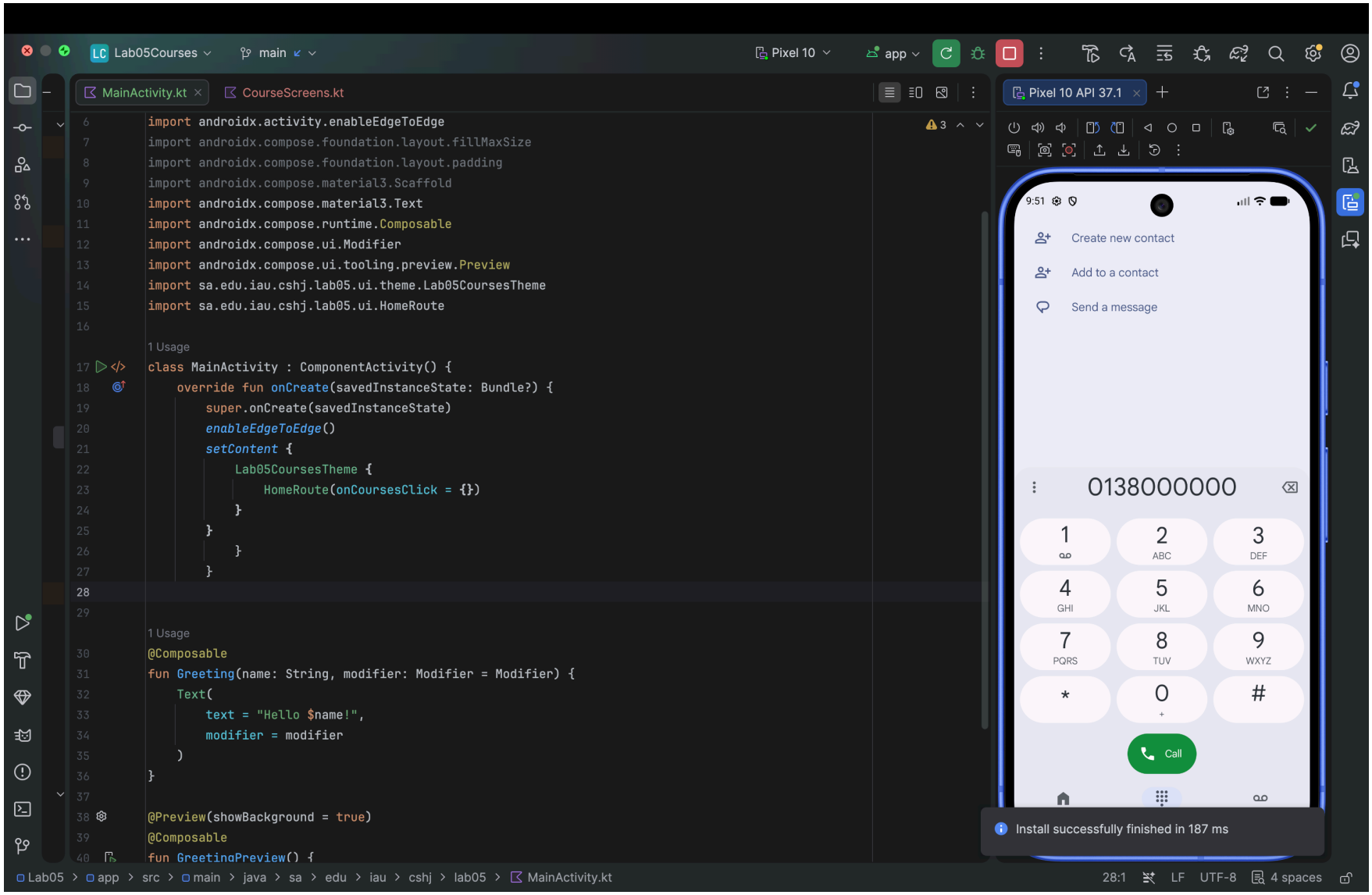
Tapping University website opens the IAU website in Chrome. The address bar shows iau.edu.sa/en.



E02. Task 1.3.2.png

E03 Task 1 Dialer intent result

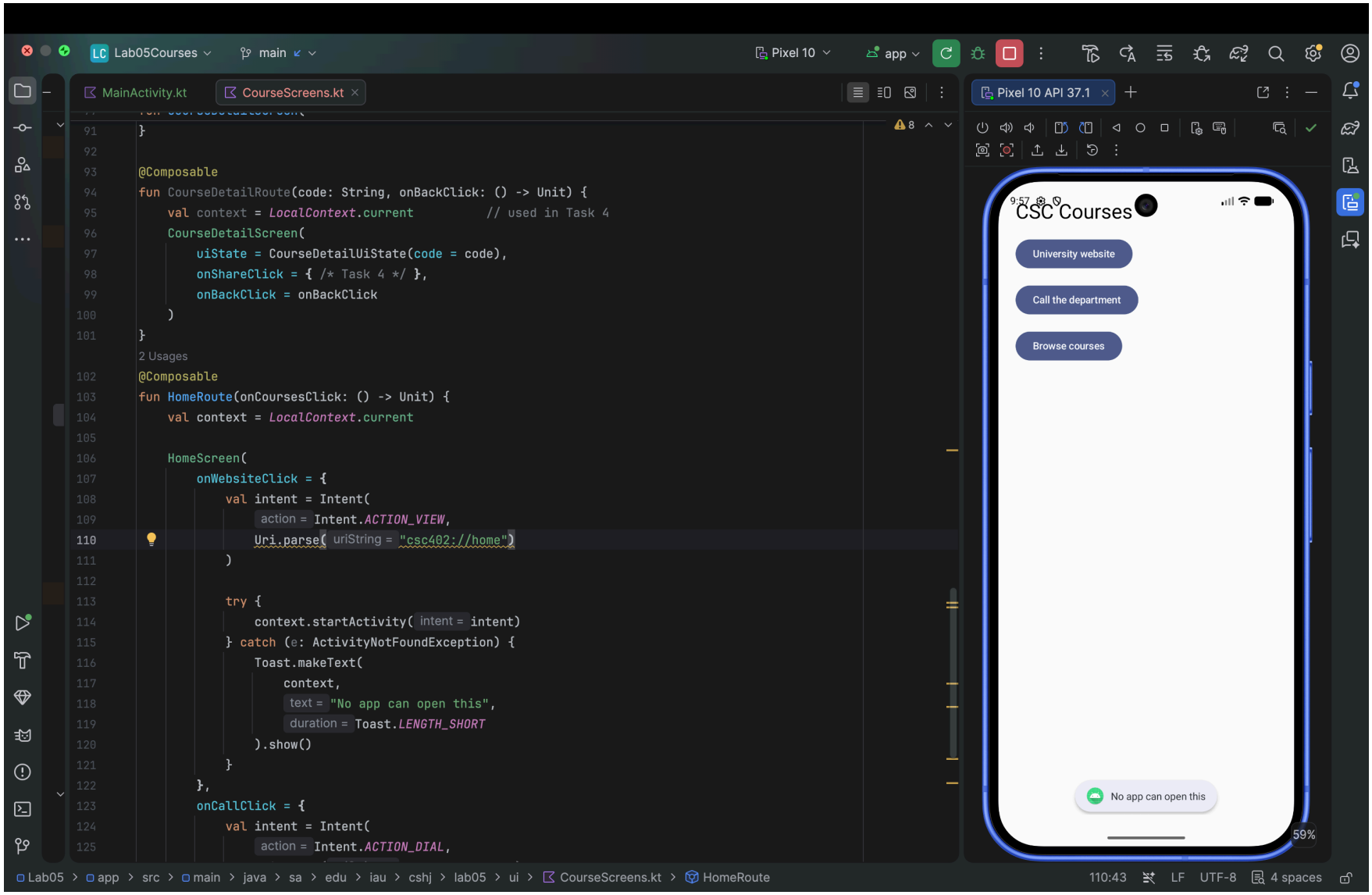
ACTION_DIAL opens the Phone dialer with the placeholder number 0138000000 already entered.



E03. Task 1.3.3.png

E04 Task 1 Unsupported URI with handling

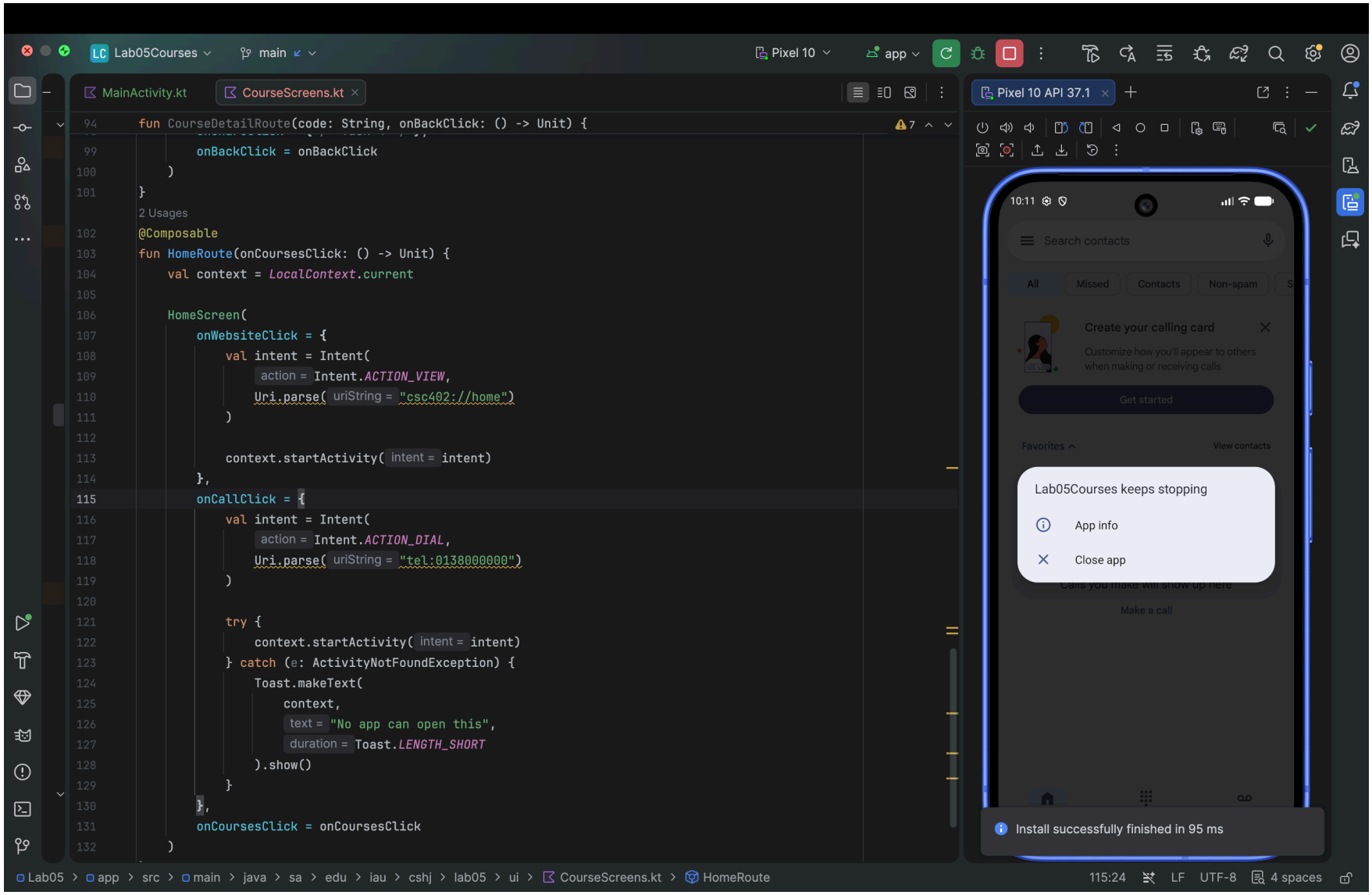
With csc402://home and try/catch in place, the app stays open and displays the Toast No app can open this.



E04. Task 1.5.png

E05 Task 1 Intentional unhandled failure

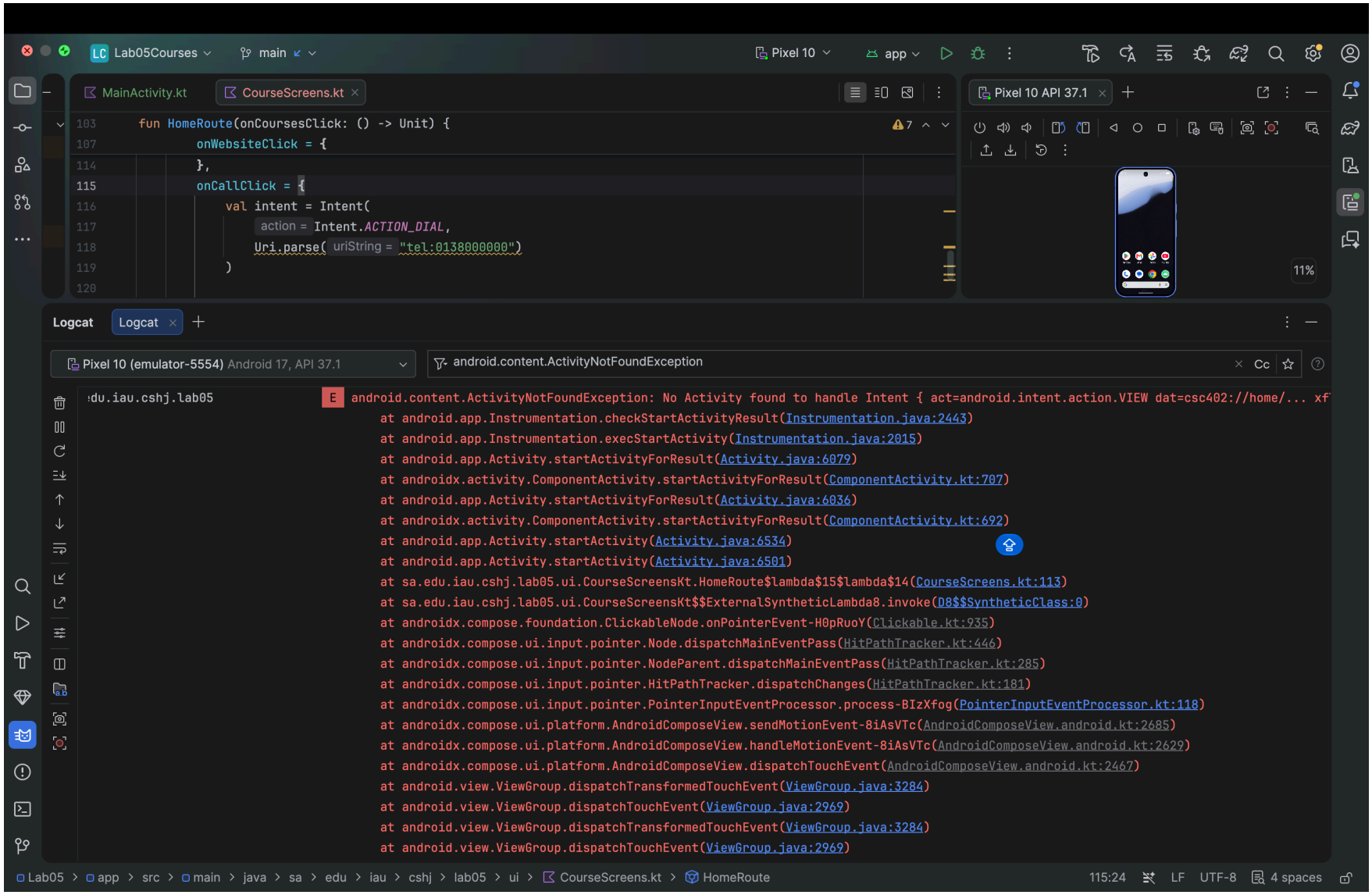
Removing the website exception handler while keeping csc402://home causes the Lab05Courses keeps stopping dialog.



E05. Task 1.6.png

E06 Task 1 ActivityNotFoundException in Logcat

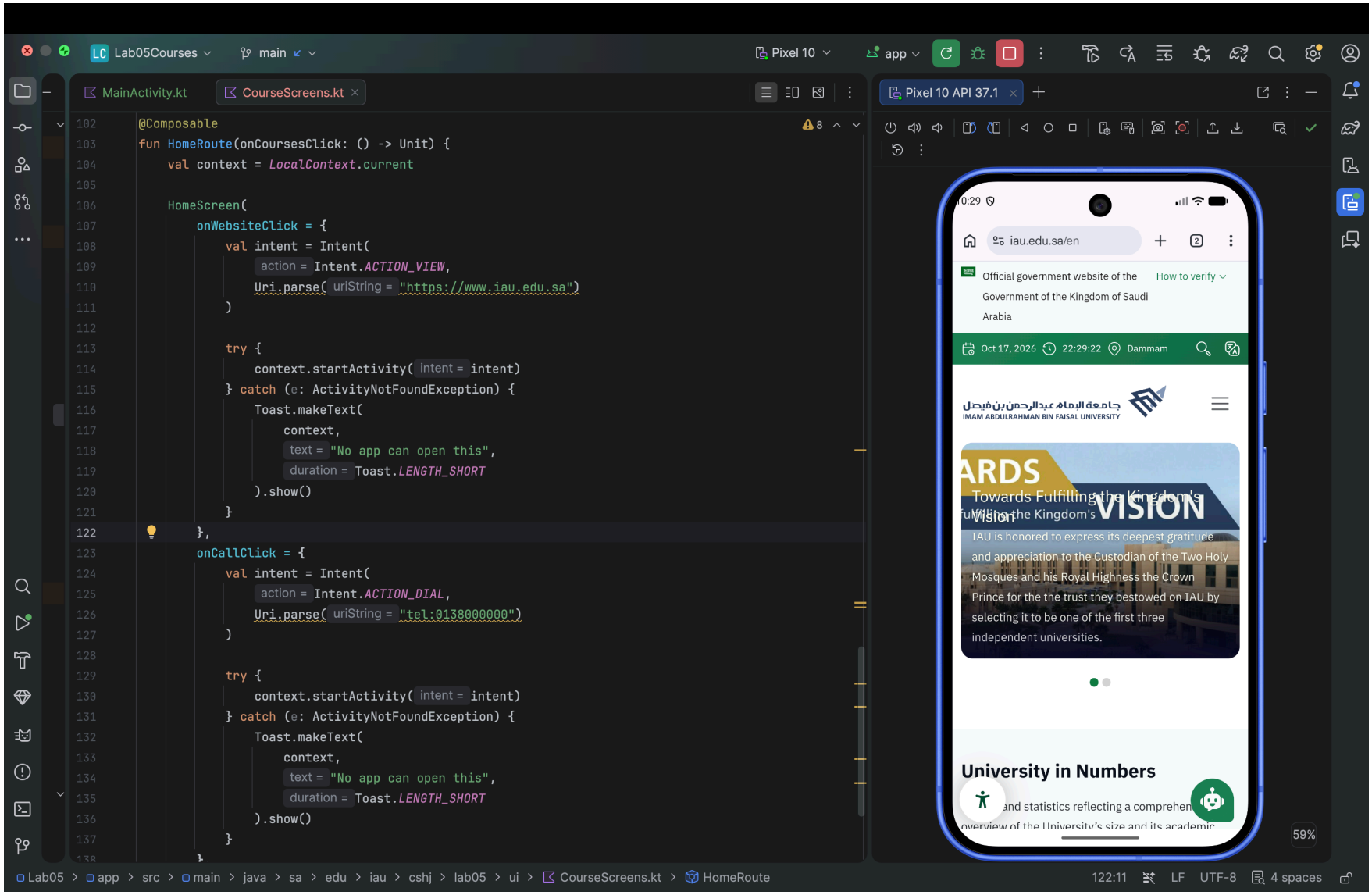
Logcat identifies ActivityNotFoundException for the ACTION_VIEW request using csc402://home. The stack points to CourseScreens.kt:113.



E06. Task 1.6.2.png

E07 Task 1 Restored website behavior

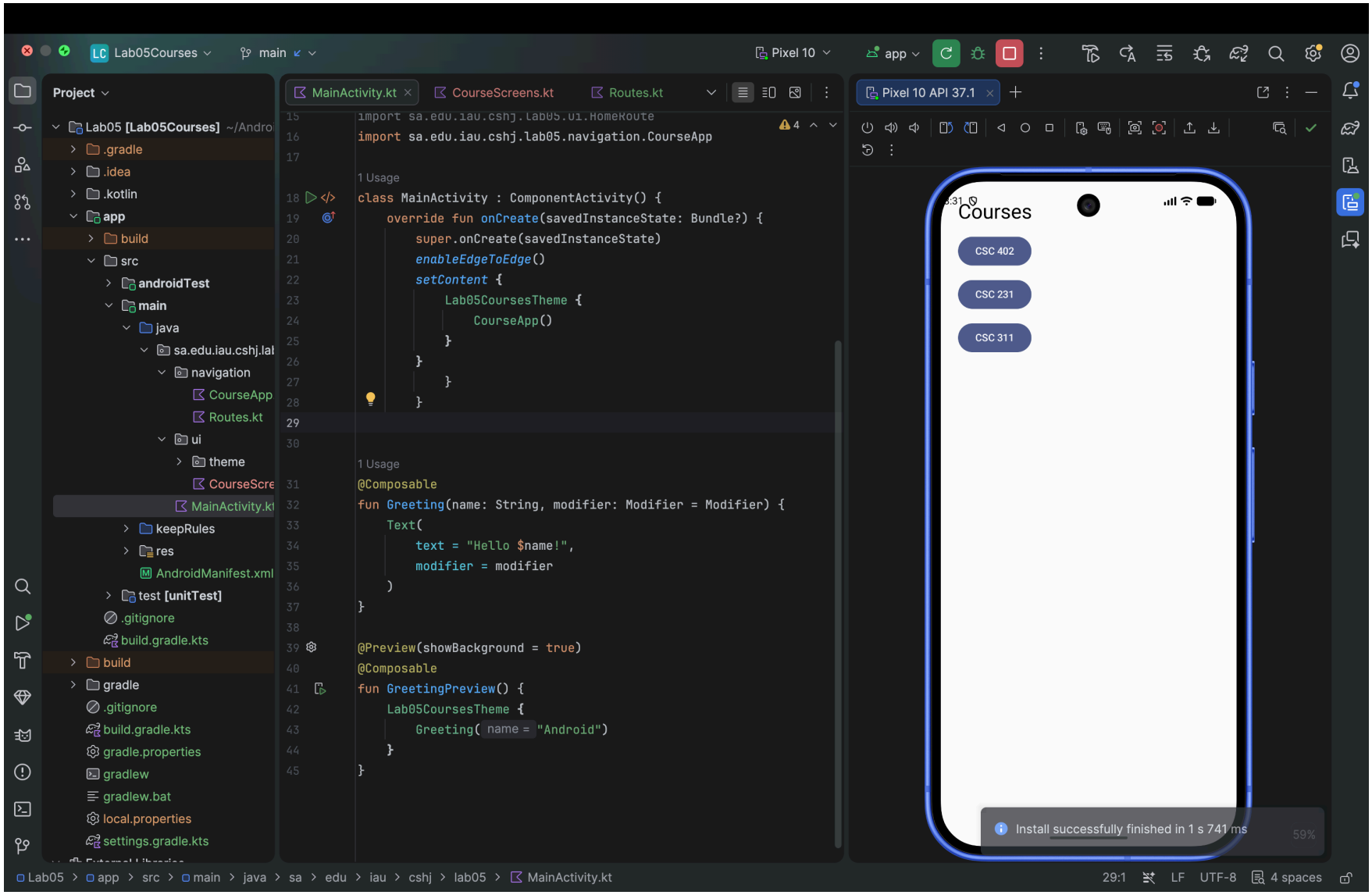
The website address is restored to HTTPS, try/catch is back in place, and the university website opens successfully.



E07. Task 1.7.png

E08 Task 3 Course list navigation

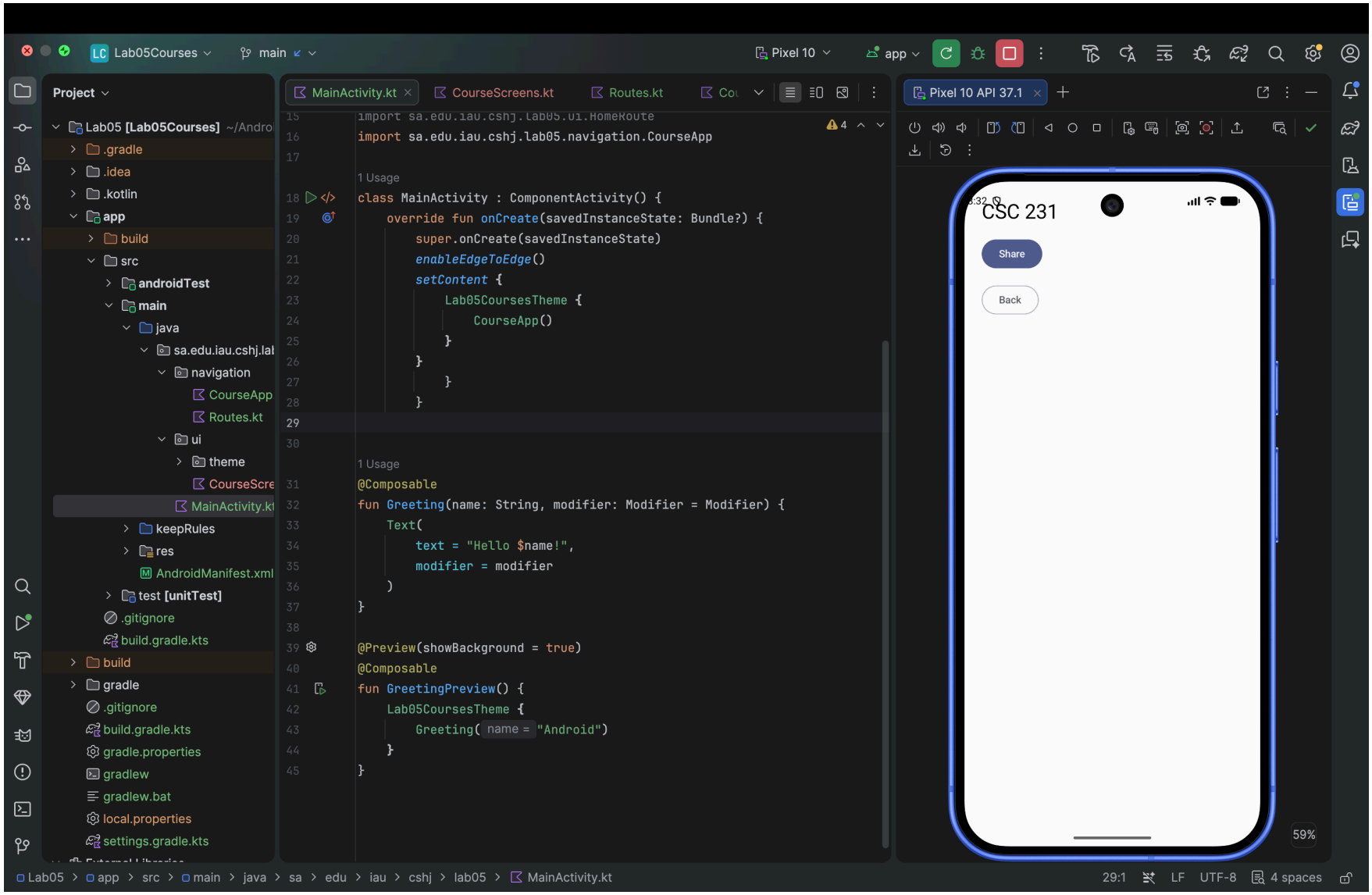
The running app displays CSC 402, CSC 231 and CSC 311. This capture records the connected course-list screen.



E08. Task 2.5.png

E09 Task 3 Selected course argument

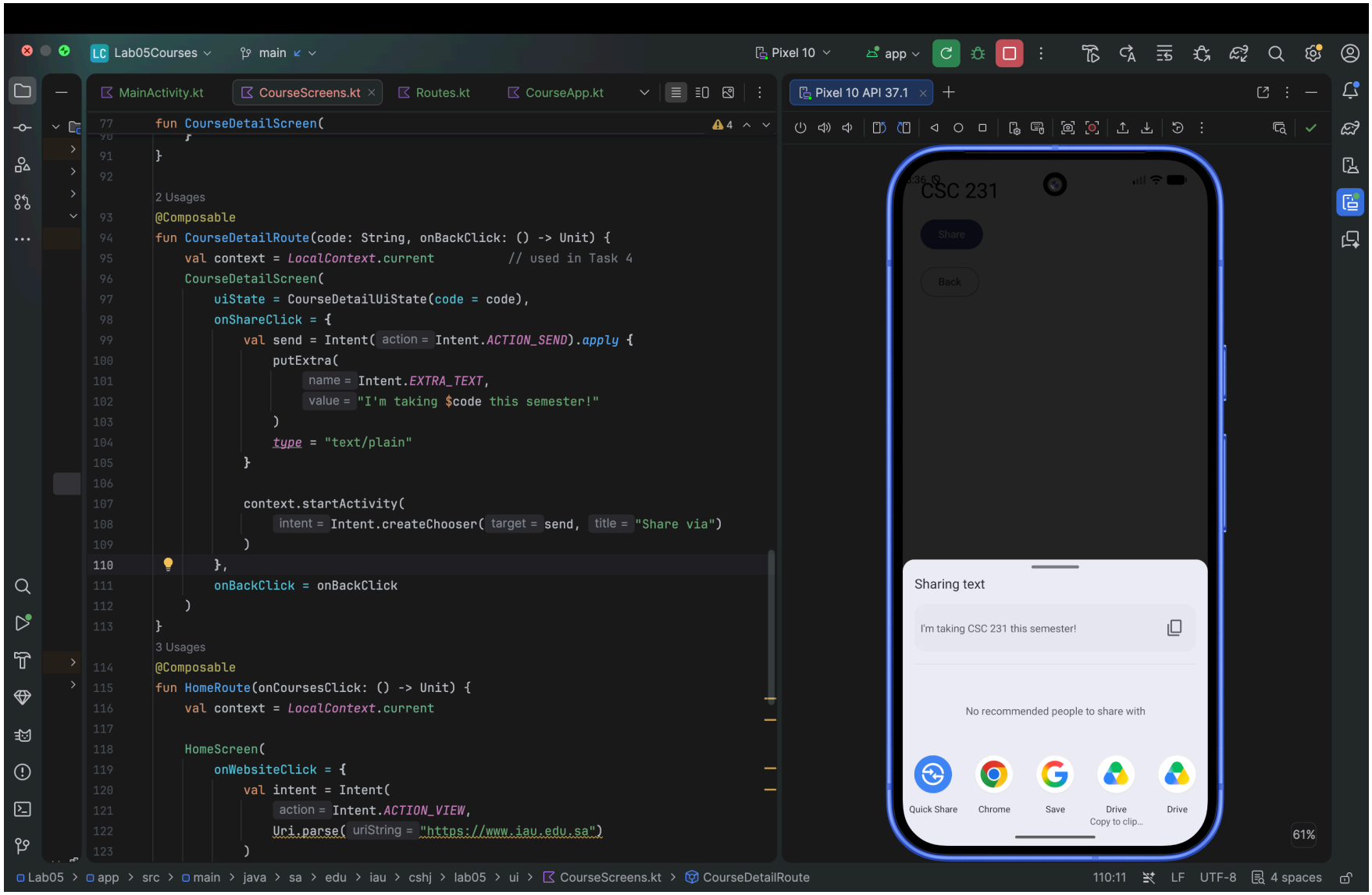
Selecting CSC 231 opens the detail screen with CSC 231 as the title and the Share and Back buttons.



E09. Task 2.5.2.png

E10 Task 4 Sharing CSC 231

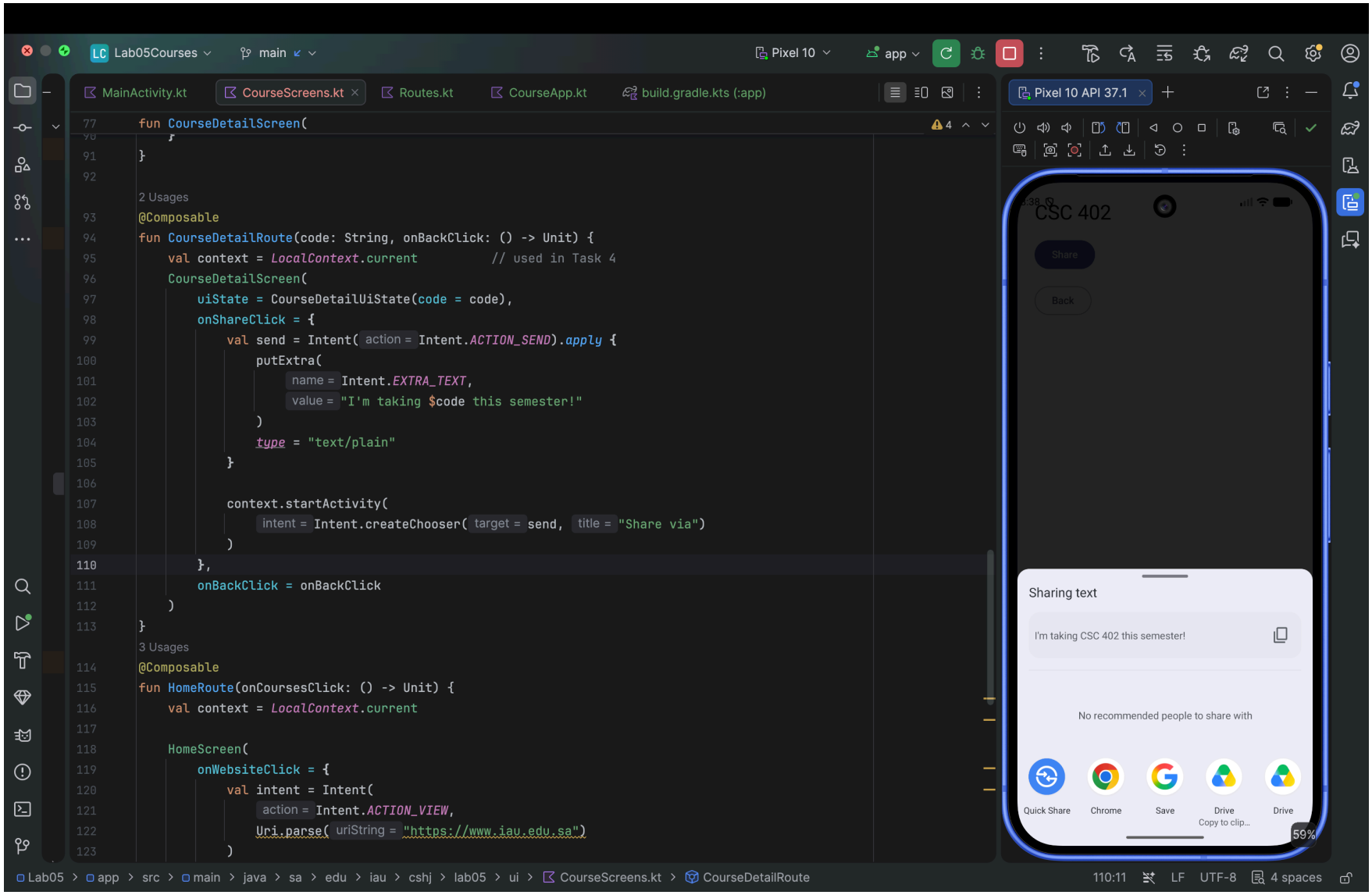
The share sheet contains I'm taking CSC 231 this semester! The Route code uses ACTION_SEND, EXTRA_TEXT and text/plain.



E10. Task 4.1.png

E11 Task 4 Sharing CSC 402

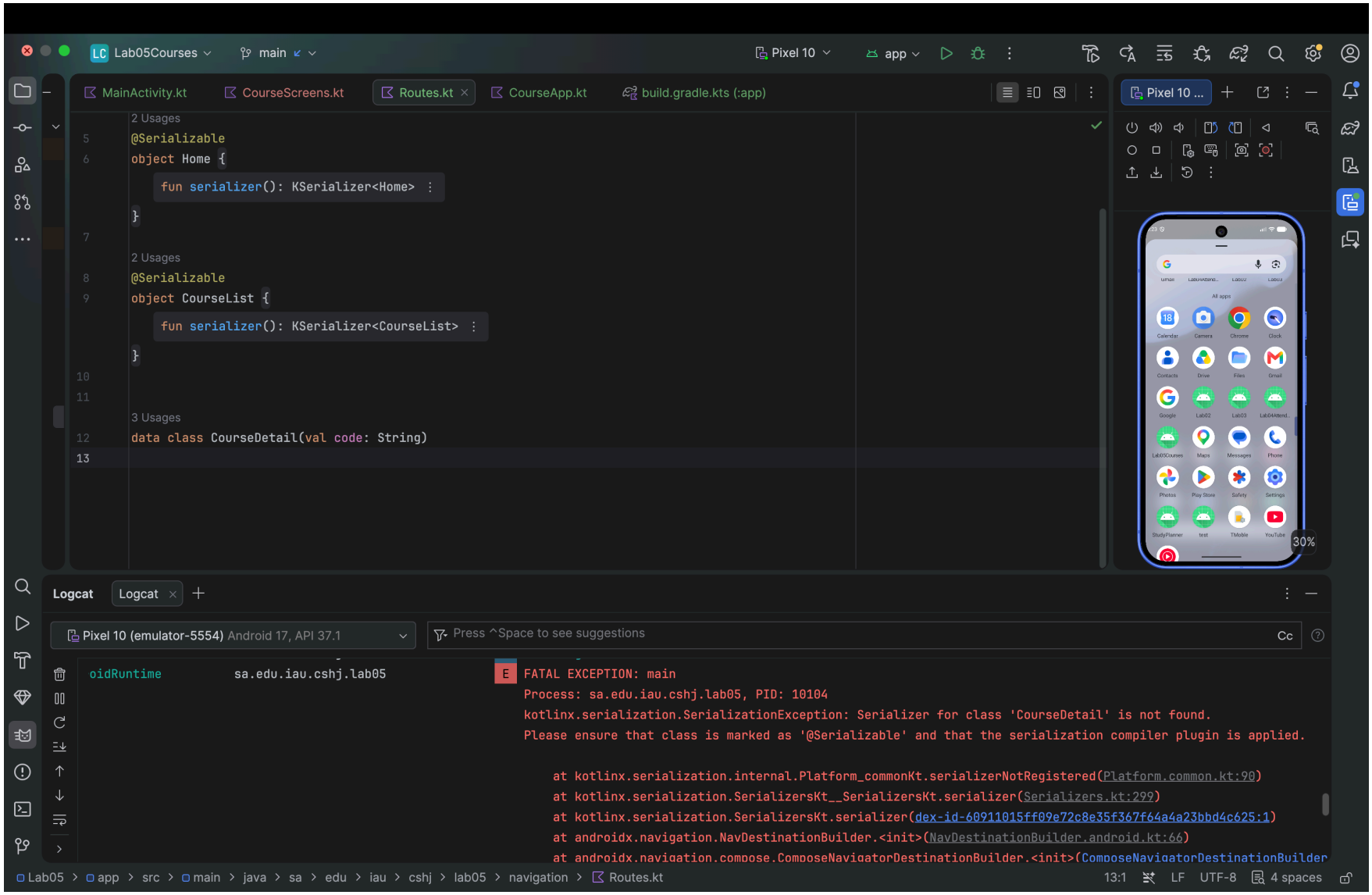
The share sheet now contains I'm taking CSC 402 this semester! Chrome and Google Drive are visible among the targets.



E11. Task 4.2.png

E12 Task 4 Intentional serialization failure

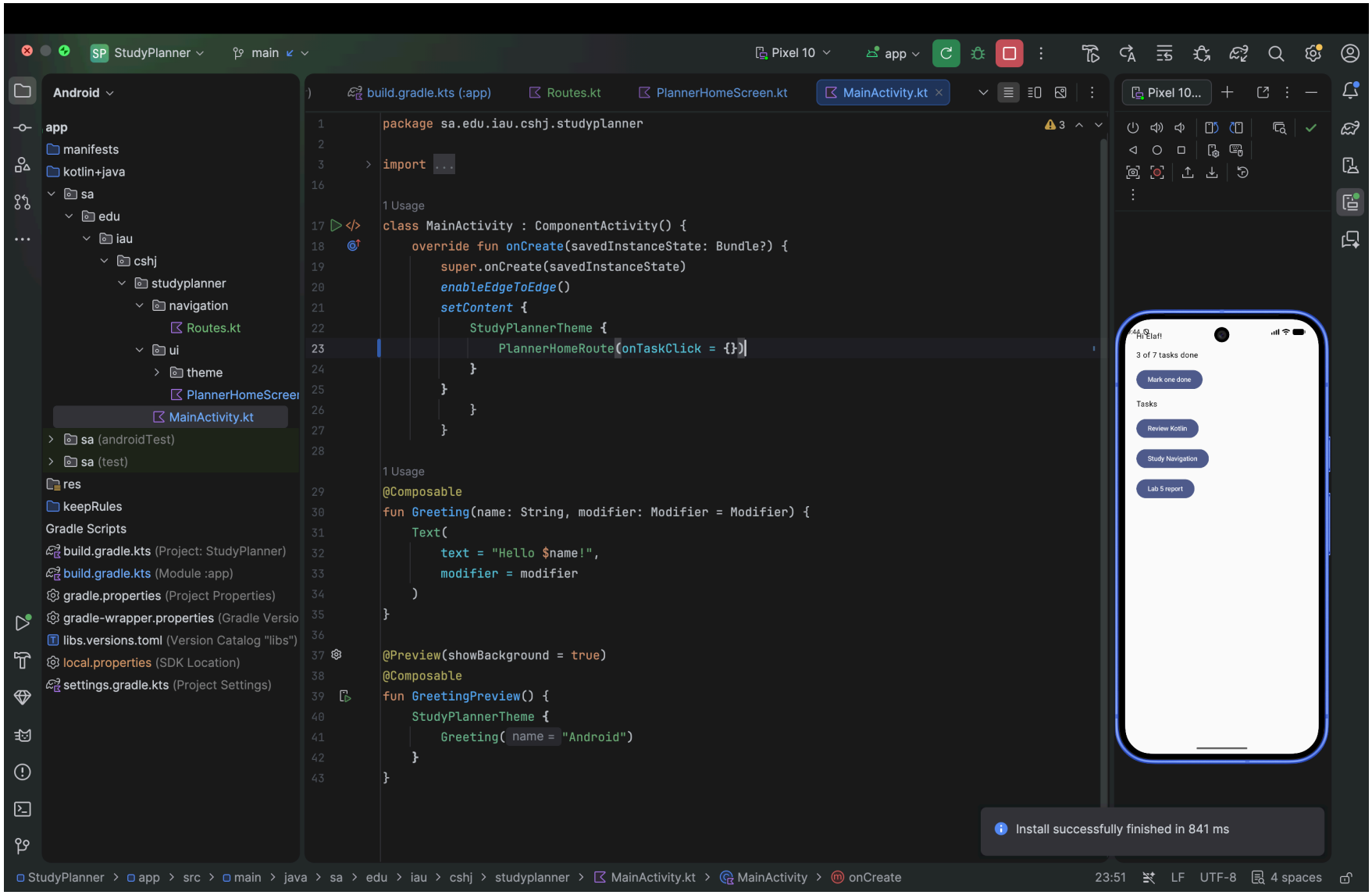
CourseDetail has no @Serializable in this test. Logcat reports that its serializer is not found. The annotation was restored afterwards.



E12. Task 4.4.png

E13 Part 2 Home task buttons

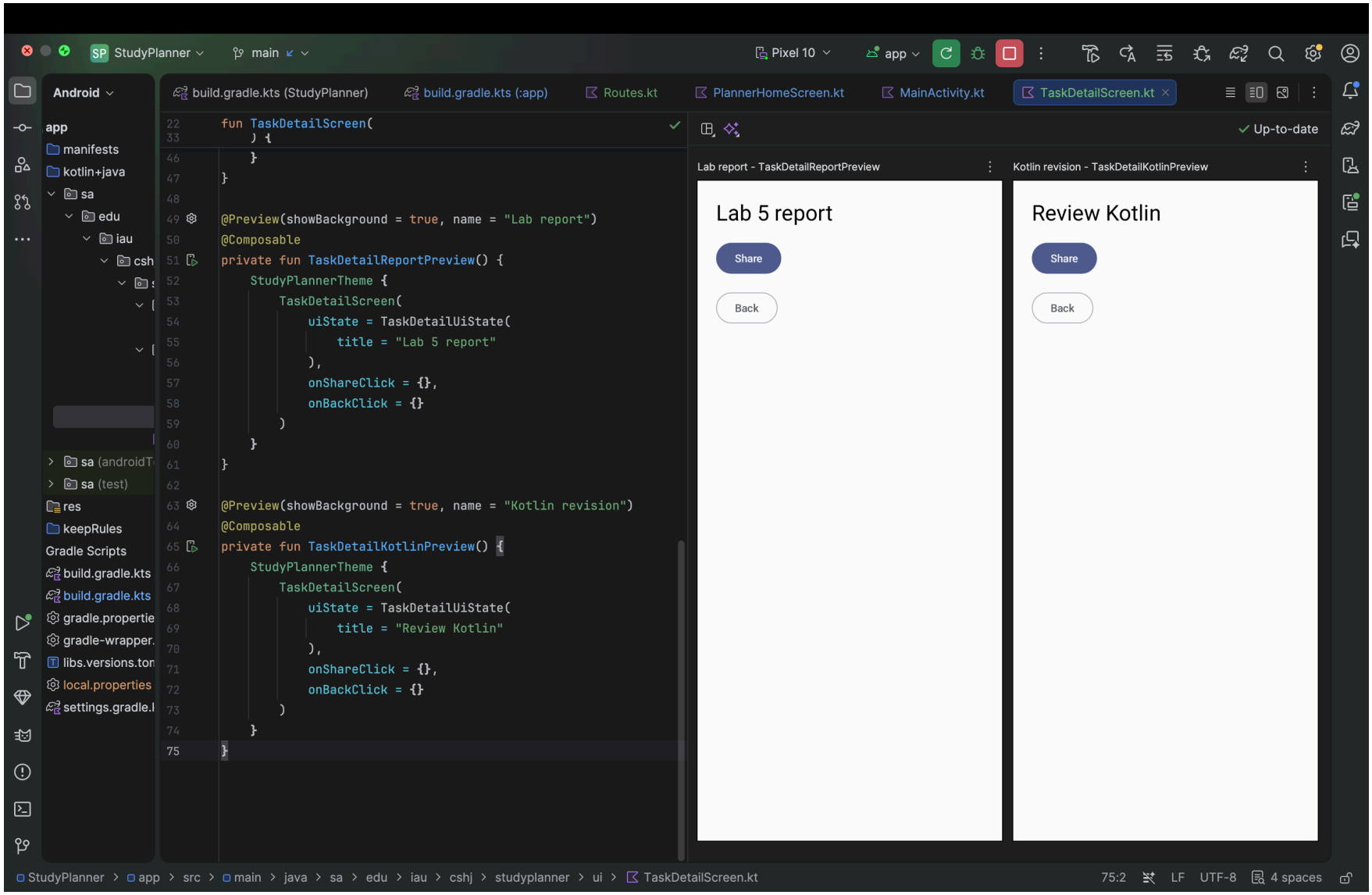
The P2.4 stage shows the greeting, progress and three task buttons. Navigation is connected in P2.7; the final Home screen is in E18.



E13. P2.4.png

E14 Part 2 Two detail previews

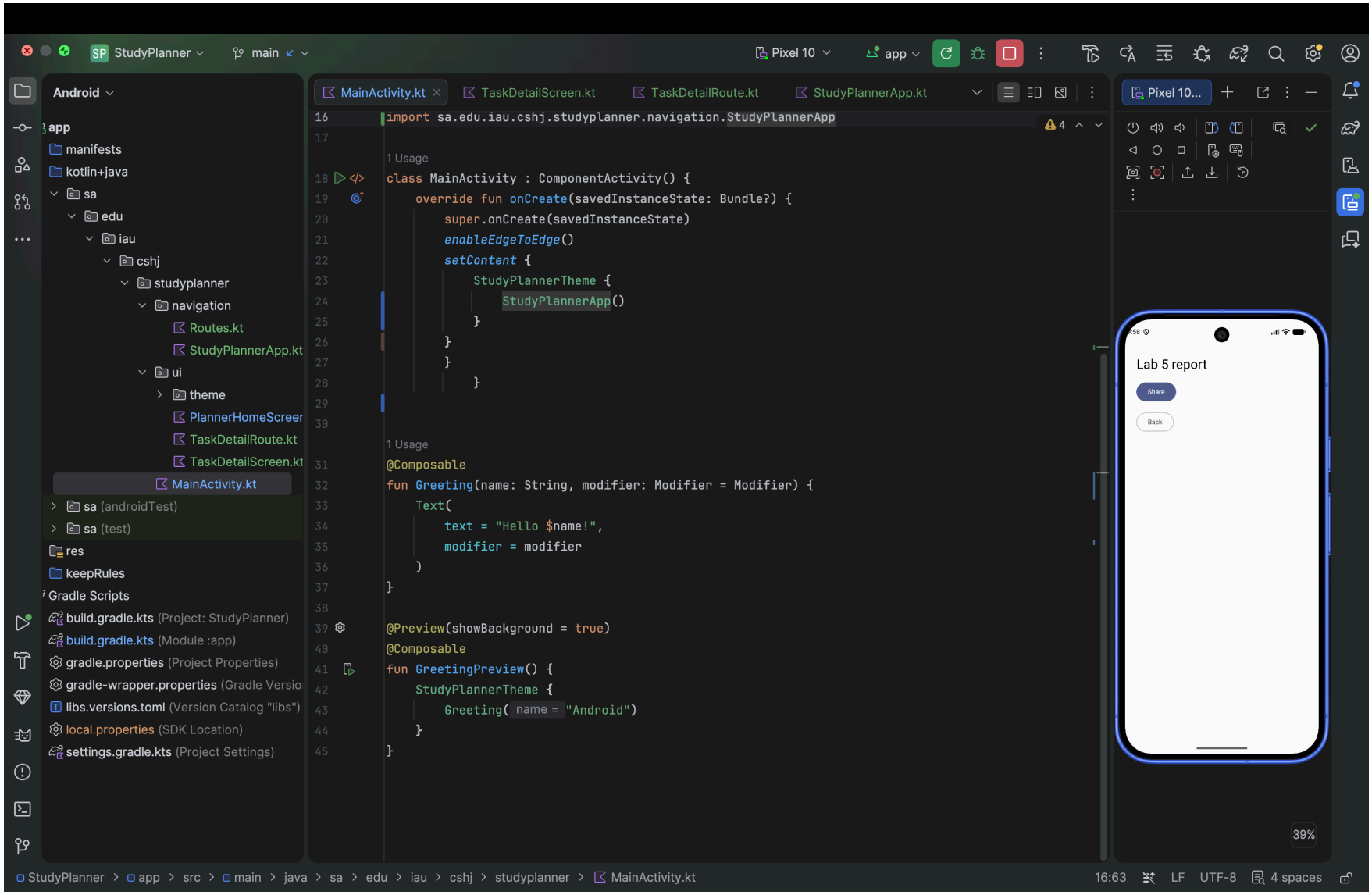
The Lab report preview shows Lab 5 report and the Kotlin revision preview shows Review Kotlin. Both use the same stateless detail composable.



E14. P2.5.png

E15 Part 2 Lab report detail

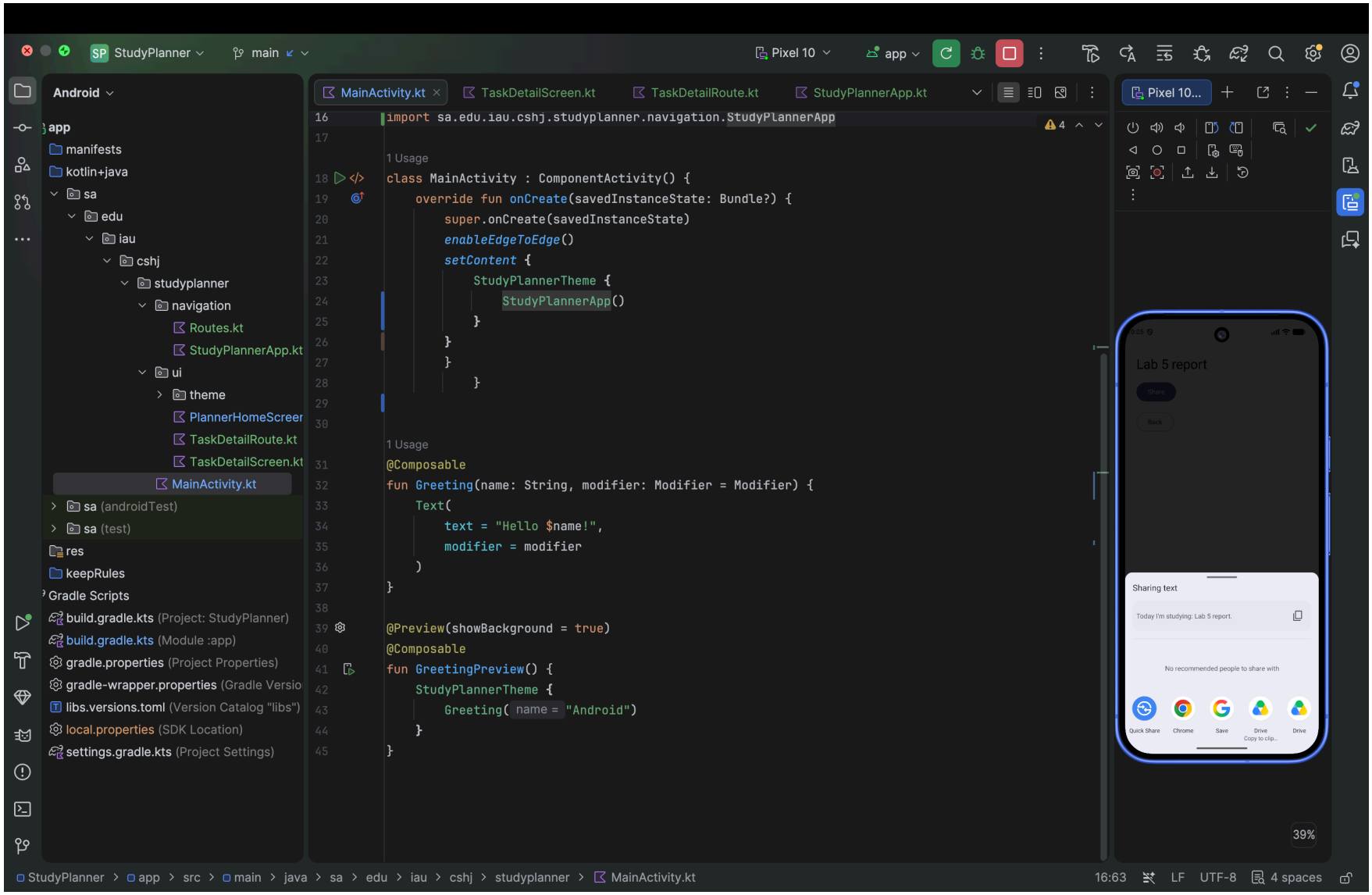
After the navigation graph is connected, tapping Lab 5 report opens a detail screen with that title, Share and Back.



E15. P2.7.png

E16 Part 2 Task share sheet

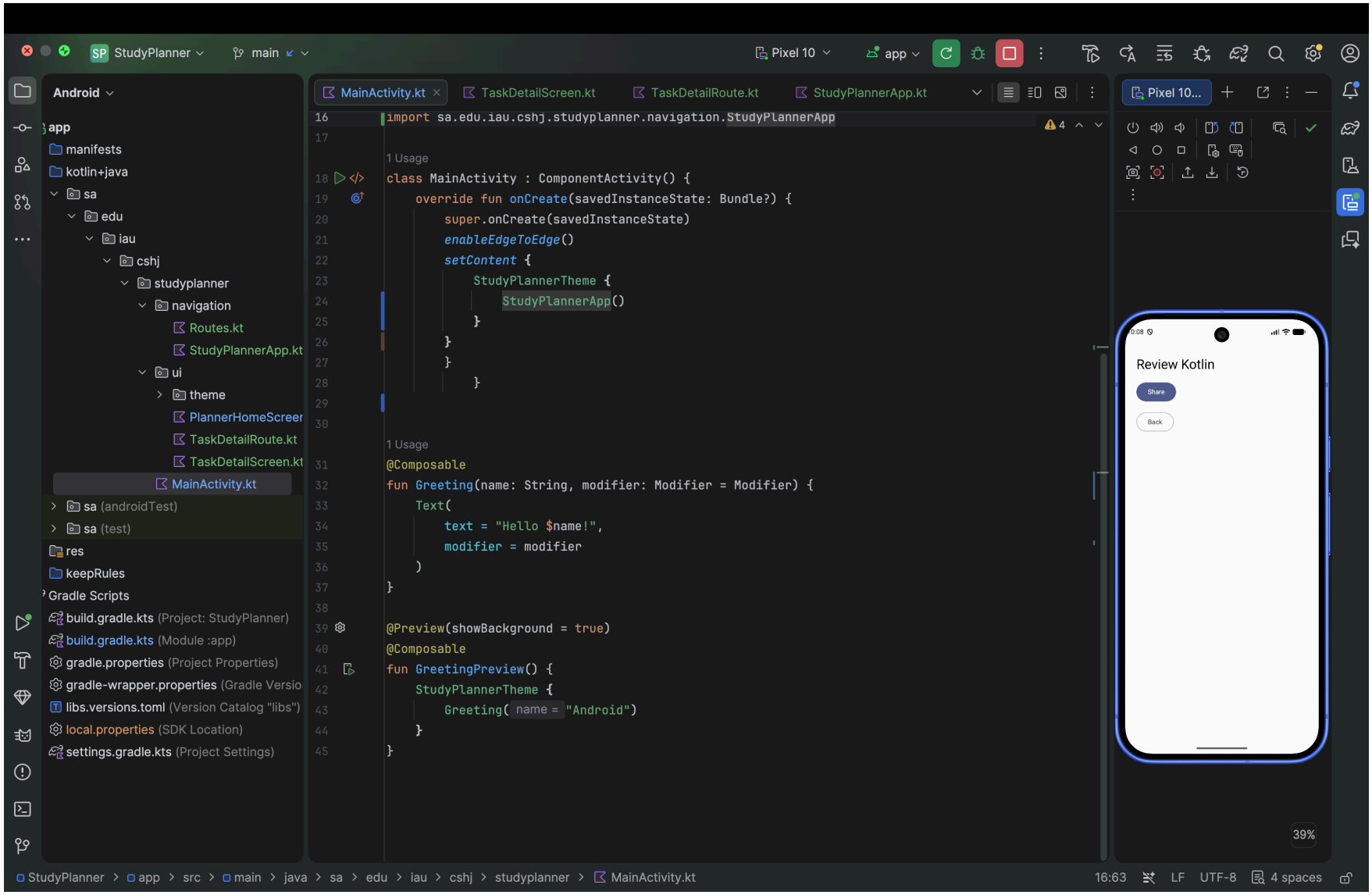
The system share sheet contains Today I'm studying: Lab 5 report. The selected title is included in the shared text.



E16. P2.7.1.png

E17 Part 2 Different selected title

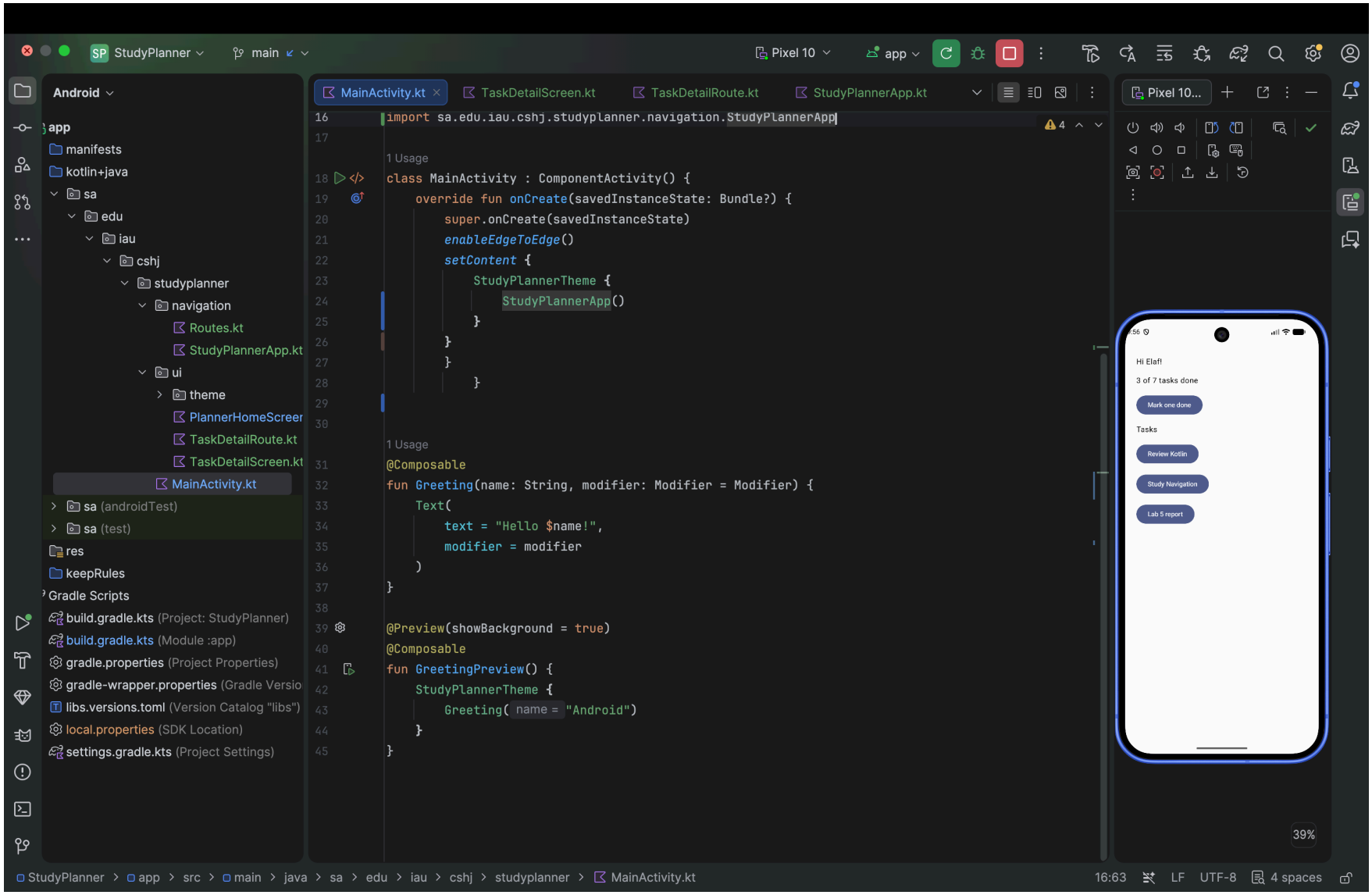
Selecting Review Kotlin displays Review Kotlin in the same detail layout, confirming that the title comes from the selected route argument.



E17. P2.7.2.png

E18 Part 2 Final connected Home

MainActivity calls StudyPlannerApp inside the theme. Home displays the task buttons below the system status area.



E18. Screenshot 1448-05-07 at 9.56.14 am.png